

AI Music Recommender — Neo Technical Deep Dive

Speaker Script + Q&A Prep

Format: 45–60 min with Igor Barakaiev. ~15–18 min prepared talk, rest is Q&A. **No AI use during the interview. No live coding.** Present from slides; keep a marker ready to sketch the loop if he digs in.

The one-sentence pitch (memorize this):

"It's an agentic music recommender over 114,000 Spotify tracks — but the interesting part isn't retrieval, it's a self-check step where Gemini re-reads its own results and catches itself agreeing with bad matches, which is the failure that quietly kills most RAG systems."

Verified numbers (say these, not the old slide numbers):

- 114,000 tracks, 125 genres
- **21 tests** (15 reliability · 4 integration · 2 unit) — 17 run with no API key and pass in ~12s
- 4 guardrails
- **3–5 Gemini calls per query** (3 clear / 4 follow-up / 5 full-retry) — NOT "2–3"
- Max score 6.5; score guardrail warns below 3.0
- 3 real bugs caught by tests before shipping

TIMING MAP (~16 min prepared)

Slides	Section	Time
1–2	Hook + before/after	2 min
3	Why RAG fails (thesis)	1.5 min
4	Architecture loop	2 min
5	The checker (CENTERPIECE)	3 min
6–8	Scoring, mood, RAG	3.5 min
9–11	Three demo cases	2.5 min
12–13	Guardrails + testing	1.5 min
14–15	Trade-offs + reflection	2 min
16	Close	15 sec

If running long, compress demos (9–11) to just Case 2. Never cut slide 5.

PER-SLIDE SCRIPT

Slide 1 — Title (20 sec)

"This is an applied-AI music recommender I built. Natural language in, grounded recommendations out, over 114,000 real Spotify tracks. I want to spend most of my time on one design decision — the self-check — because it's the part I'm proudest of and it addresses a real failure mode. Let me set up the problem first."

Slide 2 — Before/After (1.5 min)

"This started as a Module 3 project — 29 hand-crafted songs, and the user filled in genre, energy, valence as numbers by hand. A weighted formula returned the top 5. It worked, but there was no AI in it — it was a form with math behind it, and explanations were raw numbers like '+1.77 energy match.' The upgrade: you type what you want in plain language, Gemini parses it, it runs over 114k real tracks, and — the key part — an agentic loop checks its own results before answering. RAG grounds every explanation in the song's real audio features."

Slide 3 — Why RAG fails (1.5 min) — THE THESIS

"Here's the problem I actually built this around. When people say RAG fails, they usually picture retrieval breaking. But the more dangerous failure is subtler: retrieval runs fine, hands the model a batch of weak or mismatched results, and the model — because that's what LLMs do — writes a confident explanation for why they're a *perfect* match. It hallucinates agreement with bad context. And the user can't tell, because a fluent wrong answer looks exactly like a fluent right one. The system is confidently wrong, and trust erodes silently. Everything else in my design exists to catch this one thing."

Slide 4 — Architecture (2 min)

"Here's the whole flow — plan, act, check, adjust. Walk through the 5 stages: input guardrail rejects junk before it costs an API call. Gemini parses free text into a structured profile — genre, mood, energy, instrumentality, confidence. The recommender scores all 114k tracks — and this step is pure deterministic math, no AI. Then the quality check — Gemini re-reads and decides good or retry. If retry, we relax constraints and loop, capped at 3. Finally RAG explanations and a score guardrail. The mental model I'd give you: recommender.py is deliberately dumb and deterministic; agent.py is the intelligence layer wrapped around it. Gemini shows up in exactly four places."

Slide 5 — The Checker (3 min) — CENTERPIECE, GO SLOW

"This is the part I most want to talk about. After scoring, before answering, I feed Gemini the *original request* plus the top 3 results, and ask one question: do these actually match what was asked? It returns 'good' or 'retry.' Why is that separate from the scorer? Because the scorer is deterministic math — it will always rank *something* highest, even if the whole batch is weak. The checker is a second opinion from a different kind of reasoner, judging intent-match rather than score. The principle is: **the thing that generates is not the thing that approves**. That separation is the design. Mechanically: 'retry' calls `_relax_profile` which loosens constraints — attempt 1 pulls energy toward neutral, attempt 2 drops the genre requirement. Hard cap at 3, then an honest fallback. And I'll be honest about the trade-off: the checker can be *stricter* than a human. In one of my demo cases the results looked reasonable to a person but Gemini still retried. I'd rather it err strict than wave through a bad match."

[If he leans in here — stand up and sketch the loop on the whiteboard. This is the moment.]

Slide 6 — Scoring engine (1.5 min)

"Quickly on the scorer, since people ask. Deterministic weighted sum. Genre and mood get exact-match bonuses. Energy, valence, instrumentality use closeness — weight times one-minus-the-difference, so closer scores higher. Acousticness is directional. Then a diversity cap — max 2 per artist, 2 per genre, and I dedupe by artist-title pair. Max score 6.5. Same input always gives the same output — which is what makes it testable."

Slide 7 — Mood derivation (1.5 min)

"One real problem: the Spotify dataset has no mood column — just valence and energy as numbers. But users think in words like 'chill' or 'intense.' So I derive mood using the circumplex model of affect from psychology — valence times energy gives a base quadrant, then I refine with mode, tempo, and acousticness. The honest part: this is roughly 70–80% accurate on spot-checks, and the model has a western-music bias. That's in my model card. This surprised me — I didn't expect a psychology paper to end up doing my feature engineering."

Slide 8 — RAG layer (1.5 min)

"The RAG layer grounds explanations. For each pick I retrieve three things: the song's real audio features, a genre description from a JSON knowledge base, and a mood description grounded in that same circumplex model. Gemini explains using *that*, not from memory. Why static JSON and not a vector DB? I considered both, plus dynamic Gemini-generated context. At this scale — about 40 genres — JSON gives zero runtime cost, no hallucination, full control. A vector DB with embeddings and cosine search is the right *production* answer, but here it'd add infrastructure that obscures the point. That's a deliberate trade-off, not a shortcut."

Slides 9–11 — Demos (2.5 min total)

- **Case 1 (clear):** "'upbeat energetic pop to work out to' → parsed confidently, passed the check first try, 3 API calls. The explanation cites 0.89 energy — a real retrieved number."
- **Case 2 (conflict) — spend the most time here:** "'extremely high energy but deeply sad' — a contradiction. The checker failed it three times: widen energy, drop genre, then cap. And crucially it says 'I couldn't find a perfect match, here's the closest' — the score guardrail firing. It names what it can't deliver instead of faking it. 5 API calls."
- **Case 3 (vague):** "'just give me something good' — low confidence, so it asks one follow-up instead of guessing. 4 API calls because of the extra parse."

Slides 12–13 — Guardrails + testing (1.5 min)

"Four guardrails: input rejects junk pre-API, output falls back on malformed JSON, execution caps the loop at 3, score warns below 3.0. And 21 tests — 15 reliability, 4 integration, 2 unit. 17 run without any API key in about 12 seconds. They caught 3 real bugs before shipping — a type mismatch that crashed, a duplicate-track bug, and mood boundary mislabels. I tested behavior, not just code."

Slides 14–15 — Trade-offs + reflection (2 min)

"Every choice here was a trade-off — RAG over fine-tuning, JSON over a vector DB, circumplex for mood, precision over recall, a 3-retry cap. I can go deep on any of them. My biggest takeaway: system design matters more than the model. Gemini is powerful but inconsistent on its own — the loop isn't smart, it's *systematic*. And honesty beats hallucination: the 'couldn't find a match' message builds more trust than any clever explanation."

Slide 16 — Close (15 sec)

"That's the system. Happy to go deeper on the checker, the scoring math, or the retry loop — wherever you'd like."

ANTICIPATED QUESTIONS (rehearse answers out loud)

Q: Why are there two datasets in the repo? "The 29-song set is legacy from the Module 3 starter. The live system runs entirely on the 114k Spotify set — the old loader isn't called anywhere. I kept the old file only for provenance and comparison; it's effectively dead code, and I'd remove it in a cleanup." (*This is honest and disarms the question. Don't claim the old tests run on it — they don't.*)

Q: How many API calls per request, really? "Three to five. Parse is one, quality-check is one per attempt, RAG explanation is one batched call for all five songs. Clear request = 3; follow-up adds a parse = 4; full three-retry = 5."

Q: The checker uses the same model that generated — isn't that circular? "Fair challenge. It's not fully independent, but it's a different *task* — parsing/explaining is generative, checking is evaluative, and it sees only the request plus results, not its own reasoning. It reliably catches obvious mismatches. A stronger version would use a different model or a rubric, and that's a real next step."

Q: How do you know the checker actually helps? Did you measure it? "I have it in the integration tests — the conflict case triggers all three retries deterministically. What I don't have is a labeled A/B on hallucination rate; that would be the rigorous next step — a set of known-bad batches and measuring catch rate. I can describe how I'd build that." (*Honesty here scores better than overclaiming.*)

Q: Why max score 6.5? "It's the sum of the max weighted components — genre + mood + the closeness terms. 6.5 is the theoretical ceiling if every dimension matches perfectly. The 3.0 guardrail is a bit under half — below that, matches are weak enough to warn on."

Q: Circumplex accuracy — how did you get 70–80%? "Spot-checks against songs where I knew the expected mood — not a formal labeled eval, so I quote it as approximate. It's documented as a limitation, not a guarantee."

Q: What breaks at scale / what would you change for production? "Three things: swap JSON for a real vector store, add a labeled eval harness for the checker, and cache parses so repeat queries skip an API call. Also the mood derivation should be validated against human labels rather than spot-checks."

Q: Why Gemini specifically? "Free tier for a student project, fast, good structured-output support with JSON. Nothing in the architecture is Gemini-specific — the four call sites could be any capable LLM."

Q: What was the hardest bug? "The dict-vs-Song type mismatch — the recommender returned dicts but the agent expected Song objects, which crashed at runtime. A test caught it before a user ever would. It taught me to test the seams between components, not just each piece."

DELIVERY REMINDERS

- Slow down on slide 5. Silence is fine.
- When he interrupts, answer *that* question fully before returning to the deck. Don't rush back.

- If you don't know something: "I didn't measure that — here's how I'd find out." That scores better than bluffing.
- One live whiteboard sketch of the loop = worth more than any slide. Do it if he probes the checker.
- Communication is 50% of the score (per Tomas). Tight narrative > cramming detail.